Domande frequenti teoria: Basi di dati (Versione 2.0)

(Note pag. 12)

Transazioni: proprietà

Una transazione è **consistente** se: porta il sistema da uno stato valido ad un altro stato valido

Una transazione è **isolata** se: è eseguita nel sistema contemporaneamente ad altre transazioni, come se fosse l'unica ad essere eseguita

Una transazione è **permanente** se: rende permanenti le modifiche effettuate immediatamente dopo il termine della transazione (In inglese Durability)

Una transazione è **atomica** se: tutte le operazioni che la compongono sono completate, oppure sono disfatte, come se non fossero mai state eseguite

Array scalari e Array associativi in PHP

- Gli array scalari identificano ogni elemento del vettore con un numero determinato dalla sua posizione all'interno della sequenza
- Gli array associativi identificano ogni elemento del vettore con una chiave a cui l'elemento è associato in modo univoco

Esempio array scalare

```
<?php
$colori = array('bianco','rosso','verde','viola');
echo $colori[1];      //stampa rosso
?>
```

Esempio array associativo

```
<?php

$a = array(
"nome" => "Lorenzo"
"cognome" => "Vare"
"email" => "prova@gmail.com"
);

echo $a["nome"];   //stampa Lorenzo
?>
```

Rilasciato sotto licenza: **CC BY-NC 3.0**

Istruzioni SQL

L'istruzione SQL

```
CREATE TABLE T1
(A1          CHAR(5) NOT NULL,
 A2          INTEGER NOT NULL,
 A3          CHAR(5),
 PRIMARY KEY    (A1),
 FOREIGN KEY    (A2) REFERENCES T2,
 ON DELETE CASCADE);
```

Crea una tabella T1, contenente 3 attributi: A1, A2, A3
A1 è una stringa di dimensione fissa pari a 5, la quale non può essere nulla.
A2 è un intero che non può essere nullo.
A3 è una stringa di dimensione fissa pari a 5, può anche essere nulla.

Giusto per riprendere l'argomento dei domini elementari: per quanto riguarda i tipi numerici esatti, abbiamo NUMERIC, DECIMAL, INTEGER e SMALLINT.
Numeric e *Decimal* sono numeri su base decimale.

es. NUMERIC [(Precisione,Scala)] e DECIMAL [(Precisione,Scala)]
Precisione: numero totale di cifre
Scala: numero di cifre dopo la virgola

La chiave primaria è A1.
E' presente una chiave esterna, A2, la quale fa riferimento alla tabella T2.
Quando si crea una tabella dotata di chiavi esterne e con un vincolo di integrità referenziale, si possono aggiungere delle operazioni a cascata *da eseguire quanto la tabella a cui si fa riferimento viene modificata,* è molto importante non confondersi su questo.
Se noi imponiamo la condizione ON DELETE CASCADE, stiamo dicendo a parole:

"Se nella tabella esterna (T2), che contiene la tupla con il mio attributo esterno, applico una cancellazione che compromette l'integrità referenziale, allora cancella a cascata anche la tupla della tabella (T1) che conteneva come attributo referenziato, ciò che è stato cancellato."

E'un ragionamento un po' contorto ma va ricordato. (Aggiungo un esempio:)
T2 -> AUTORI(CodAutore)
T1 -> LIBRI(CodLibro, CodAutore)
Se cancelli un autore (CodAutore) allora cancella anche i suoi libri (CodLibro,CodAutore)

Rilasciato sotto licenza: **CC BY-NC 3.0**

Istruzioni PHP

L'istruzione PHP:

```
foreach ($var1 as $var2) {
  echo "$var2 <br />";
}
```

La risposta corretta : è corretta se $var1 è un array scalare

Come funziona il foreach in PHP?

Ci sono due sintassi differenti per il foreach, una applicabile agli array associativi, un'altra agli array scalari.

Sintassi array scalari

```
<?php
$colori = array('bianco','rosso','verde','viola');

foreach ($colori as $value) {
        echo "$value <br />";
}

?>
```

Nota bene: la sintassi è diversa rispetto a Java:

- Java: for(String str : vettoreStr)
- PHP: foreach(vettore : valore)

Sintassi array associativi ->

fonte: slide corso DBMG Politecnico di Torino

```
<?php
  $anno = array("Gennaio" => 31,
                "Febbraio" => 28,
                "Marzo" => 31,
                "Aprile" => 30,
                "Maggio" => 31,
                "Giugno" => 30,
                "Luglio" => 31,
                "Agosto" => 31,
                "Settembre" => 30,
                "Ottobre" => 31,
                "Novembre" => 30,
                "Dicembre" => 31);

  foreach ($anno as $mese => $giorni) {
    echo "$mese ha $giorni giorni. <br />";
  }
?>
```

Gennaio ha 31 giorni.
Febbraio ha 28 giorni.
Marzo ha 31 giorni.
Aprile ha 30 giorni.
Maggio ha 31 giorni.
Giugno ha 30 giorni.
Luglio ha 31 giorni.
Agosto ha 31 giorni.
Settembre ha 30 giorni.
Ottobre ha 31 giorni.
Novembre ha 30 giorni.
Dicembre ha 31 giorni.

Rilasciato sotto licenza: **CC BY-NC 3.0**

*Istruzione SQL (2): attenzione al **NO ACTION** nei vincoli di integrità referenziale*

L'istruzione SQL

```
CREATE TABLE T1
(A1 CHAR(5) NOT NULL,
A2 INTEGER,
A3 CHAR(5) NOT NULL,
PRIMARY KEY (A1, A3),
FOREIGN KEY (A2) REFERENCES T2,
ON UPDATE NO ACTION);
```

La **risposta corretta** è:
crea una tabella T1 in cui si richiede che venga vietata l'esecuzione di ogni operazione di UPDATE su T2 che violi l'integrità referenziale

Non confonderti con questa **risposta sbagliata**:
crea una tabella T1 in cui a seguito di un'operazione di UPDATE su T2 che violi l'integrità referenziale non viene intrapresa alcuna azione

Funzioni MySQL per PHP

MySQLi (MySQL improved) è un'estensione di PHP che consente di interfacciarsi a DB MySQL

mysqli_connect(): in caso di successo restituisce un identificativo di connessione MySQL, in caso di insuccesso restituisce FALSE

die(): arresta l'esecuzione dello script e stampa un messaggio
mysqli_connect_errno(): restituisce il codice dell'errore di connessione
mysqli_connect_error(): restituisce l'errore di connessione

mysqli_close(): chiude il collegamento con il DBMS e rilascia le relative risorse
Prende come parametro opzionale l'identificativo della connessione

Rilasciato sotto licenza: **CC BY-NC 3.0**

mysqli_query():
Richiede come parametro l'id della connessione e la query da eseguire, in formato stringa
In caso di successo restituisce il risultato della query, in caso di insuccesso restituisce FALSE

esempio:

$con = mysqli_connect('localhost', 'Lorenzo', 'xyz', 'nomeDb');
$sql = "SELECT Matricola FROM Studenti WHERE ..."

$result = mysqli_query($con, $sql);

mysqli_error(): restituisce il testo dell'errore relativo alla funzione Mysql eseguita più recentemente

```
if ($result==false)
        die('Errore: ' .  mysqli_error($con));
```

Esecuzione preparata
mysqli_prepare()

- Richiede come parametri l'identificativo di connessione e la query da eseguire, in formato stringa
- I parametri all'interno della query sono indicati con un '?'
- La funzione invia la query a MySQL che ne controlla la validità e ne verifica la correttezza
- In caso di successo restituisce un oggetto di tipo mysqli_stmt, in caso di insuccesso restituisce FALSE

Binding dei parametri
Prima di eseguire la query bisogna collegare ciascun parametro con la variabile corrispondente (operazione di 'binding´)

mysqli_stmt_bind_param()

- Richiede come parametri l'oggetto restituito da msqli_prepare(), il tipo dei dati e le variabili che devono essere assegnate ai parametri della query
- In caso di successo restituisce TRUE, in caso di insuccesso restituisce FALSE

$stmt = mysql_prepare($con, "INSERT INTO Prodotti VALUES(?,?)");
Binding dei parametri -> mysqli_stmt_bin_param($stmt, "si", "Scarpa", "25");

Rilasciato sotto licenza: **CC BY-NC 3.0**

Come secondo parametro del binding c'è una codifica un po' contorta:

- "s" per stringa
- "i" per intero
- "d" per numero reale

Se si si ha -> mysqli_stmt_bin_param($stmt, "sddi", …)
Significa che al posto di quei 3 punti la funzione si aspetta: una stringa, un numero reale seguito da un altro numero reale ed infine un numero intero.

$CodProdotto = "S34FF"
$Qty = 34
$Taglia = 40
$Prezzo = 27.30

mysqli_stmt_bin_param($stmt, "sddi", $CodProdotto,$Qty,$Taglia,$Prezzo);

Esecuzione della query preparata

mysqli_stmt_execute()
Richiede come parametro l'oggetto restituito da mysqli_prepare()
In caso di successo restituisce TRUE, in caso di insuccesso restituisce FALSE

(esempio: mysqli_stmt_execute($stmt), dove $stmt contiene la query preparata, e il binding è già avvenuto grazie alla funzione di binding)

```
//preparazione della query
$stmt = mysqli_prepare($con, "SELECT Provincia FROM Citta WHERE nome=?");
if (!stmt)
        die ('Query error; ' .mysqli_error());

$nome= "Torino";

//binding dei parametri
mysqli_stmt_bind_param($stmt, "s", $nome);

//esecuzione della query
mysqli_stmt_execute($stmt);
```

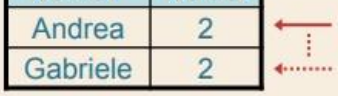
fonte: slide corso DBMG Politecnico di Torino

Rilasciato sotto licenza: **CC BY-NC 3.0**

Lettura del risultato

- Il risultato della funzione mysqli_query() viene memorizzato in una variabile di tipo "resource"
  - Una variabile speciale, che contiene il riferimento ad una risorsa esterna
- La lettura del risultato avviene riga per riga: ciclo che prevede due fasi
  - Acquisizione di una riga della tabella (utilizzo di un cursore)
  - Lettura della riga acquisita

NomeF	NSoci
Andrea	2
Gabriele	2



DBMG

88

Acquisizione di una riga

- Esistono diverse possibilità per acquisire una riga della tabella
- Chiamata alla funzione mysqli_fetch_row()
  - Richiede come parametro la risorsa restituita da mysqli_query()
  - Restituisce l'array corrispondente alla riga corrente, o FALSE nel caso in cui non ci siano righe disponibili
  - Ciascuna colonna del risultato viene memorizzata in un elemento dell'array, a partire dall'indice "0"

```
while ($row = mysqli_fetch_row($result)) {
      ...
      }
```

DBMG

89

fonte: slide corso DBMG Politecnico di Torino

Rilasciato sotto licenza: **CC BY-NC 3.0**

Istruzione HTML

L'istruzione HTML:

```
<select name="corso">
  <option value="23ACIPL">Analisi matematica II</option>
  <option value="08CKRPL">Statistica</option>
  <option value="14AFQPL" selected>Basi di dati</option>
</select>
```

Note

- Le select producono menu a tendina
- Il valore selezionato è quello indicato con selected
- il value è il valore che verrà passato al programma che elabora la richiesta dell'utente

Proprietà errata

La transazione ha proprietà X se:

non permette di eseguire operazioni che violino i vincoli che limitano l'accesso ai dati per ragioni di sicurezza

non esiste questa proprietà tra quelle che abbiamo studiato, riguardo alle transazioni

Gestione dei permessi

L'istruzione SQL

```
GRANT INSERT
ON VIEW V
TO ROSSI, VERDI;
```

(a) autorizza gli utenti Rossi e Verdi a inserire dati nella vista V
(b) nessuna delle risposte è corretta
(c) è scorretta in ogni caso, perchè non si possono mai inserire dati attraverso una vista
(d) autorizza solo il DBA a inserire dati nella vista V
(e) è scorretta poiché bisogna specificare anche la colonna su cui effettuare l'inserimento

Rilasciato sotto licenza: **CC BY-NC 3.0**

Flusso operazioni applicazione PHP che interagisce con una base dati

1) Connessione al DBMS
2) Selezione della base di dati
3) Invio della query
4) Ricezione del risultato
5) Chiusura della connessione
6) Elaborazione del risultato

Istruzione SQL 3 - Unique

L'istruzione SQL

```
CREATE TABLE T1
(A1            CHAR(5) NOT NULL,
A2             INTEGER UNIQUE,
A3             CHAR(5) NOT NULL,
PRIMARY KEY    (A1));
```

Crea una nuova tabella base T1, con chiave primaria A1 e in cui l'attributo A2 potrebbe essere scelto alternativamente come chiave primaria se fosse anche NOT NULL

Unique = valore che non può essere ripetuto ma che può essere nullo
Chiave primaria = valore che non può essere né nullo né ripetuto

Quiz sulle funzioni PHP sqli - (viste precedentemente)

Data l'istruzione PHP (in cui nella variabile $risultato è stato memorizzato il risultato di una query SQL):

```
$var = mysql_fetch_row($risultato);
```

(a) $var è un array associativo

(b) $var è un array scalare

(c) nessuna delle risposte è corretta

(d) $var è un numero intero

(e) si può accedere agli elementi dell'array $var sia con un indice numerico sia con la chiave corrispondente al nome dell'attributo della tabella restituita

Rilasciato sotto licenza: **CC BY-NC 3.0**

Ripasso delle funzioni per l'acquisizione del risultato di una query

Lettura del risultato

- Il risultato della funzione mysqli_query() viene memorizzato in una variabile di tipo "resource"
    - Variabili resource: variabili di tipo speciale che contengono un riferimento ad una risorsa esterna

La lettura del risultato avviene riga per riga: ciclo che prevede due fasi

- Acquisizione di una riga della tabella (utilizzo di un cursore)
- Lettura riga acquisita

Acquisizione di una riga: esistono diverse possibilità per acquisire una riga della tabella

1) Chiamata alla funzione: mysqli_fetch_row()
    - Richiede come parametro la risorsa restituita da mysqli_query()
    - Resistuisce l'array corrispondente alla riga corrente, o FALSE nel caso in cui non ci siano righe disponibili
    - Ciascuna colonna del risultato viene memorizzata in un elemento dell'array (scalare), a partire dall'indice "0"

```
while($row = mysqli_fetch_row($result)) {
        ...
}
```

2) Chiamata alla funzione: mysqli_fetch_assoc()
    - Richiede come parametro la risorsa restituita da mysqli_query()
    - Restituisce l'array associativo corrispondente alla riga corrente, o FALSE nel caso in cui non ci siano righe disponibili
    - Ciascuna colonna del risultato viene memorizzata in un elemento dell'array associativo in corrispondenza alla chiave definita dal nome del campo
    - Non viene definito un indice numerico

3) Chiamata alla funzione: mysqli_fetch_array()
    - E' la funzione più generale
    - Richiede come parametro la risorsa restituita da mysqli_query() e il tipo di array da riempire (scalare, associativo o entrambi)
    - MYSQLI_ASSOC: l'array risultante è di tipo associativo
    - MYSQLI_NUM; l'array risultante è di tipo scalare
    - MYSQLI_BOTH: l'array risultante è accessibile sia con indice numerico sia con chiave corrispondente al nome del campo

Rilasciato sotto licenza: **CC BY-NC 3.0**

Funzione utile: numero di righe di una risorsa (“resource”)

int mysqli_num_rows($risorsa)

Restituisce il numero di righe della risorsa, oppure FALSE in caso di insuccesso

esempio:

```
if( mysqli_num_rows($result)<=0) {
        echo “<h5>Nessun risultato</h5>”;
}
else{
… //accedi alle righe
}
```

Altre funzioni:

int mysqli_num_fields(resource $result)

- Restituisce il numero di campi (attributi) della risorsa $result, o FALSE in caso di insuccesso

```
for ($i = 0; $i < mysqli_num_fields($result); $i++){
  $title  =  mysqli_fetch_field($result);
  $name = $title->name;
  echo "<th> $name </th>";
}
```

string mysqli_fetch_field(resource $result)

- Restituisce la prossima colonna come oggetto. Per ottenerne il nome occorre selezionarne la proprietà ‘name´

```
if( mysqli_num_rows($result) > 0 ){
  //Intestazione tabella
  echo "<table border=1 cellpadding=10>";
  echo "<tr>";
  for ($i = 0; $i < mysqli_num_fields($result); $i++){
    $title  =  mysqli_fetch_field($result);
    $name = $title->name;
    echo "<th> $name </th>";
  }
  echo "</tr>";
  // riempimento tabella
  while ($row = mysqli_fetch_row($result)) {
  ...
  }
```

Rilasciato sotto licenza: **CC BY-NC 3.0**

Nota per il lettore

E' molto improbabile che queste poche pagine bastino a coprire gli argomenti che verranno richiesti durante il quiz teorico all'esame.
Non lo dico per fare allarmismo ma basti guardare la differenza tra la sessione di luglio 2020 e settembre 2020 per accorgersi dell'uscita di un quiz relativo alla gestione dei privilegi*, un argomento che non era mai uscito negli appelli precedenti.
Tuttavia ci si accorge di come le domande si ripetono, infatti nell'appello successivo (febbraio), è tornata una domanda sullo stesso argomento.
Questo era solo un esempio, ce ne sono diversi.

Dunque non bisogna stupirsi se in questo appello verrà fatta una domanda sulle viste, argomento di teoria che in 4 appelli non è ancora uscito.

Il mio consiglio è di riguardare le parti di teoria più utilizzabili per la creazione di domande:

SQL fondamenti
- Gestione delle tabelle

SQL avanzato
- Gestione viste
- Gestione transazioni
- SQL per le applicazioni
- Controllo dell'accesso
- Gestione indici

Programmazione WEB
- Linguaggio PHP
- MySQL per PHP
- + Un po' di HTML

Buono studio! *L. V.*

*(Gestione dei privilegi -> slide "Controllo dell'accesso" - SQL Avanzato)

Rilasciato sotto licenza: **CC BY-NC 3.0**

Gestione delle tabelle

CREATE TABLE NomeTabella
(NomeAttributo Dominio [ValoreDiDefault] [Vincoli]
{ , NomeAttributo Dominio [ValoreDiDefault] [Vincoli]}
AltriVincoli);

Domini elementari: CHAR, VARCHAR, BIT, NUMERIC, DECIMAL, INTEGER, SMALLINT, FLOAT, REAL, DOUBLE PRECISION, INTERVAL

Intervalli temporali: INTERVAL PrimaUnitàDiTempo [TO UltimaUnitàDiTempo]
es. INTERVAL year TO month
es. INTERVAL day TO second

Tempo: TIMESTAMP [(Precisione)] [WITH TIME ZONE]

Definizione di **domini personalizzati**:
CREATE DOMAIN NomeDominio AS TipoDiDato [ValoreDiDefault] [Vincolo]
esempio:

```
CREATE DOMAIN Voto AS SMALLINT
    DEFAULT NULL
    CHECK (Voto >= 18 and Voto <=30)
```

Operazioni concesse dall **ALTER TABLE**

- aggiunta di una nuova colonna
- definizione di nuovo valore di default per una colonna (attributo) esistente
- eliminazione di una colonna (attributo) esistente
- definizione di un nuovo vincolo di integrità
- eliminazione di un vincolo di integrità esistente

RESTRICT: l'elemento non viene rimosso se è presente in altre tabelle
CASCADE: tutti gli elementi che dipendono da un elemento rimosso vengono rimossi

Esempi:

```
ALTER TABLE FORNITORI
ADD COLUMN NDipendenti SMALLINT; --aggiungi colonna

ALTER TABLE FORNITORI
DROP COLUMN NSoci RESTRICT;  --rimuovo la colonna Nsoci (restrict)

ALTER TABLE FP ALTER COLUMN Qta SET DEFAULT 0; --cambio val default
```

Rilasciato sotto licenza: **CC BY-NC 3.0**

Operazione **DROP TABLE**

- Tutte le righe della tabella sono eliminate insieme alla tabella

RESTRICT: la tabella non è rimossa se è presente in qualche definizione di tabella, vincolo o vista
CASCADE: se la tabella compare in qualche definizione di vista anche questa è rimossa

Dizionario dei dati - Contiene metadati relativi a…

- descrizione di tutte le strutture (tabelle, indici, viste) della base di dati
- stored procedure SQL
- privilegi degli utenti
- statistiche
    - sulle tabelle della base di dati
    - sugli indici della base di dati
    - sulle viste della base di dati
    - sulla crescita della base di dati

Oracle: 3 collezioni di informazioni per il dizionario

- USER_*: metadati relativi ai dati dell'utente corrente
- ALL_*: metadati relativi ai dati di tutti gli utenti
- DBA_*: metadati delle tabelle di sistema

I *vincoli di integrità* sono:

- definiti nelle istruzioni CREATE o ALTER TABLE
- memorizzati nel dizionario dati di sistema

Tipologie di **vincoli di tabella**

- chiave primaria
- ammissibilità del valore nullo
- unicità
- vincoli generali di tupla

Chiave primaria:

- un solo attributo: NomeAttributo Dominio PRIMARY KEY
- uno o più attributi: PRIMARY KEY (ElencoAttributi)

Ammissibilità del valore nullo:

- NULL non ammesso: NomeAttributo Dominio NOT NULL

Rilasciato sotto licenza: **CC BY-NC 3.0**

Unicità:

- Un attributo o un insieme di attributi non può assumere lo stesso valore in righe diverse della tabella
- un solo attributo: NomeAttributo Dominio UNIQUE
- per uno o più attributi: UNIQUE (ElencoAttributi)
- È ammessa la ripetizione del valore NULL (considerato sempre diverso)

Chiave candidata:

- La chiave candidata è un insieme di attributi che potrebbe assumere il ruolo di chiave primaria
- La combinazione UNIQUE NOT NULL permette di definire una chiave candidata che non ammette valori nulli
    - NomeAttributo Dominio UNIQUE NOT NULL

Vincoli generali di tupla:

- NomeAttributo Dominio CHECK (Condizione)
- possono essere indicati come condizione i predicati specificabili nella clausola WHERE

Chiave esterna:

FOREIGN KEY (ElencoAttributiReferenzianti)
REFERENCES NomeTabella [(ElencoAttributiReferenziati)]

Politiche di gestione dei vincoli:

Esempio: Referenziata-> Dipartimenti e Referenziante->Impiegati

Impiegati (Matr, NomeI, Residenza, *DNum*)
Dipartimenti (DNum, DNome, Sede)

Operazioni di modifica o cancellazione dalla tabella **referenziata (Dipartimenti)** causano sulla tabella **referenziante (Impiegati)**:

- CASCADE: propagazione dell'operazione di aggiornamento o cancellazione
- SET NULL/DEFAULT: null o valore di default in tutte le colonne delle tuple che hanno valori non più presenti nella tabella referenziata
- NO ACTION: non si esegue l'azione invalidante

Rilasciato sotto licenza: **CC BY-NC 3.0**

Gestione delle viste

Teoria:

- il contenuto è definito mediante un'interrogazione SQL sulla base di dati
- il contenuto non è memorizzato fisicamente nella basi di dati
- la vista è utilizzabile nelle interrogazioni come se fosse una tabella

Esempio di creazione:

```
CREATE VIEW PICCOLI_FORNITORI AS
SELECT CodF, NomeF, NSoci, Sede
FROM F
WHERE Nsoci<3;
```

Alternativa senza viste:

```
SELECT *
FROM F
WHERE NSoci<3;
```

Vantaggi uso delle viste:

- scomposizione di un'interrogazione complessa in sottointerrogazioni associate alle viste
- in assenza di table function, alcune tipologie di interrogazioni possono essere definite solo mediante l'uso di viste
- Sicurezza: è possibile introdurre meccanismi di protezione della privatezza diversi per ogni utente o gruppo
- in caso di ristrutturazione di una base di dati, è possibile definire viste che corrispondono a tabelle eliminate

Aggiornabilità delle liste:

- sono aggiornabili le viste in cui una sola riga di ciascuna tabella di base corrisponde a una sola riga della vista

Non sono aggiornabili viste che nel blocco più esterno dell'interrogazione che la definisce:

- non contiene la chiave primaria della tabella su cui è definita
- contiene join che rappresentano corrispondenze uno a molti o molti a molti
- contiene funzioni aggregate
- contiene DISTINCT

Rilasciato sotto licenza: **CC BY-NC 3.0**

Esempio n.4: vista non aggiornabile

```
CREATE VIEW FORNITORI_IMPORTANTI (CodF, NomeF) AS
SELECT DISTINCT CodF, NomeF
FROM   F, FP
WHERE F.CodF=FP.CodF AND
        Qta>100;
```

- La vista non è aggiornabile
  - è presente un join
  - è presente la parola chiave DISTINCT

DBMG 47

Esempio n.4: vista modificata

```
CREATE VIEW FORNITORI_IMPORTANTI (CodF, NomeF) AS
SELECT CodF, NomeF
FROM   F
WHERE CodF IN (SELECT CodF
                FROM   FP
                WHERE Qta>100);
```

- La vista è aggiornabile
  - il join è stato realizzato mediante IN
  - la parola chiave DISTINCT non è più necessaria

DBMG 48

Esempio n.2: modifica

- Modifica in NUMSOCI_SEDE di
  (2, 'Torino') in (3, 'Milano')

- più tuple sono associate alla coppia (2, 'Torino')
  - quale tupla deve essere modificata in F?

⇒ Non è aggiornabile

Rilasciato sotto licenza: **CC BY-NC 3.0**

Viste: **Check option**

```
CREATE VIEW PRODOTTI_TAGLIA_MEDIA_O_GRANDE(CodP,
NomeP, Taglia) AS
SELECT CodP, NomeP, Taglia
FROM P
WHERE Taglia>=42
WITH CHECK OPTION;
```

- La vista è aggiornabile
  - non si possono aggiornare le tuple presenti nella vista con valori di taglia minori di 42

Oracle: Viste che descrivono i dati creati da un utente

- USER_TABLES contiene metadati relativi alle tabelle dell'utente
- USER_TAB_STATISTICS contiene le statistiche calcolate sulle tabelle dell'utente
- USER_TAB_COL_STATISTICS contiene le statistiche calcolate sulle colonne delle tabelle dell'utente

Rilasciato sotto licenza: **CC BY-NC 3.0**

Gestione delle transazioni

Sistema transazionale: sistema che mette a disposizione un meccanismo per la definizione e l'esecuzione di transazioni

Proprietà transazioni (clicca qui - pag. 1) Nota: sono molto richieste all'esame

START TRANSACTION (solitamente implicito)

Comandi per definire la fine di una transazione: (Azione corrispondente)

- COMMIT [WORK] Transazione terminata con successo (commit)
- ROLLBACK [WORK] Transazione terminata con insuccesso (abort)

Rilasciato sotto licenza: **CC BY-NC 3.0**

Controllo dell'accesso

- Esempi di risorsa: tabella, vista, dominio, procedura, singolo attributo...
- Qualsiasi componente dello schema di una base di dati è una *risorsa*
- Le risorse sono protette mediante la definizione di ***privilegi di accesso***

Informazioni caratterizzanti un privilegio:
- la risorsa a cui si riferisce
- tipo di privilegio
- utente che concede il privilegio
- utente che riceve il privilegio
- facoltà di trasmettere il privilegio ad altri utenti

Tipi di privilegio: INSERT, UPDATE, DELETE, SELECT, REFERENCES, USAGE

Tipi di privilegio e risorse

Tabelle e viste: INSERT, UPDATE, DELETE, SELECT, REFERENCES(solo tabella)
Attributi: UPDATE, REFERENCES

USAGE: permette di utilizzare la risorsa (per esempio, un nuovo tipo di dato) nella definizione di nuovi schemi

Privilegi del creatore della risorsa

Solo il creatore della risorsa ha il privilegio di eliminare una risorsa (DROP) e modificarne lo schema (ALTER)

Utente system: l'amministratore del sistema (utente system)
possiede tutti i privilegi su tutte le risorse

Concessione dei privilegi: GRANT

GRANT ElencoPrivilegi ON NomeRisorsa TO ElencoUtenti
[WITH GRANT OPTION]

WITH GRANT OPTION: facoltà di trasferire il privilegio ad altri utenti

ALL PRIVILEGES: parola chiave da inserire al posto di "ElencoPrivilegi" per fare riferimento a tutti i privilegi

Rilasciato sotto licenza: **CC BY-NC 3.0**

Rimozione dei privilegi: REVOKE

REVOKE ElencoPrivilegi ON NomeRisorsa FROM ElencoUtenti
[RESTRICT|CASCADE]

Il comando REVOKE può togliere:
- tutti i privilegi che erano stati concessi
- un sottoinsieme dei privilegi concessi

RESTRICT: il comando non deve essere eseguito qualora la revoca dei privilegi all'utente comporti qualche altra revoca di privilegi (opzione di default)

CASCADE: revoca anche tutti i privilegi che erano stati propagati

Per ogni privilegio revocato sono:
- revocati in cascata tutti i privilegi concessi
- rimossi tutti gli elementi della base di dati che erano stati creati sfruttando questi privilegi

RUOLO: è un profilo di accesso
-> definito dall'insieme di privilegi che lo caratterizzano

Ogni utente gode dei privilegi associati al ruolo.

Vantaggi nell'uso del ruolo:
- controllo dell'accesso più flessibile (es. assegnazione di ruoli multipli)
- semplificazione dell'attività di amministrazione
    - possibilità di definire un profilo di accesso in un momento diverso dalla sua attivazione

Ruoli in SQL-3:

`CREATE ROLE` NomeRuolo

istruzione `GRANT` -> Definizione dei privilegi di un ruolo e del ruolo di un utente

`SET ROLE NomeRuolo` -> Associazione dinamica di un ruolo a un utente

Rilasciato sotto licenza: **CC BY-NC 3.0**

Gestione degli indici

Intro:

- All'interno di un DBMS relazionale, i dati sono rappresentati come collezioni di record memorizzati in uno o più file
- Non esiste un'organizzazione fisica dei dati che sia efficiente per qualunque tipo di lettura e scrittura dei dati

//Esempio proposto a lezione riguardo alla tabella dipendenti e la loro residenza

Soluzione per evitare la scansione completa della tabella:

- Organizzazione fisica progettata specificamente per l'interrogazione proposta
- File con struttura sequenziale ordinata

Pro e contro

- Vantaggi: si evita la lettura sequenziale dell'intero file
- Svantaggi:
    - mantenere l'ordinamento è computazionalmente oneroso
    - organizzazione fisica efficiente per un tipo di interrogazioni, *non è efficiente quando occorre eseguire altre tipologie di interrogazioni*

È possibile definire **strutture fisiche accessorie** che permettano di facilitare l'accesso ai dati.

Vantaggi e svantaggi delle strutture fisiche accessorie:

(Vantaggi)

- si evita la lettura completa e sequenziale del file
- accesso diretto solo ai record di interesse
- il costo di mantenimento della struttura accessoria è meno oneroso rispetto al costo di mantenimento del file con struttura ordinata

(Svantaggi)

- occupazione di spazio maggiore: è necessario spazio supplementare per memorizzare la struttura fisica accessoria
- la struttura accessoria può essere utilizzata solo quando nell'interrogazione compare lo specifico attributo su cui si basa la struttura
    - possono essere necessarie strutture accessorie per più attributi o combinazioni di attributi

Rilasciato sotto licenza: **CC BY-NC 3.0**

Indici: strutture fisiche accessorie offerte dai DBMS relazionali per migliorare l'efficienza delle operazioni di accesso ai dati

- Istruzioni per la gestione degli indici non fanno parte dello standard SQL

Le *strutture fisiche di accesso* descrivono il modo in cui i dati sono organizzati in memoria secondaria per garantire operazioni di ricerca e modifica dei dati efficienti. (Tipologie: sequenziale, albero, accesso calcolato).

Struttura sequenziale

- Disposizione sequenziale dei record in memoria secondaria
- Blocchi di memoria consecutivi nel file
- La sequenza dei record dipende dal valore assunto da un campo di ordinamento, detto chiave di ordinamento

Struttura ad albero

- Una struttura ad albero permette di raggiungere l'insieme delle locazioni fisiche dei record corrispondenti al valore prescelto del campo chiave
- la locazione fisica di un record indica la posizione fisica del record all'interno del file in memoria secondaria
- Esempio: B-tree, B^+-tree

Struttura ad accesso calcolato

- Accesso associativo efficiente ai dati, basato sul valore di un campo chiave
- Richiede un algoritmo di calcolo per localizzare il blocco fisico del file contenente i record corrispondenti al valore del campo chiave
- Esempio: struttura hash

Indici in SQL

CREATE INDEX NomeIndice ON NomeTabella (ElencoAttributi)

- L'ordine in cui compaiono gli attributi in ElencoAttributi è importante

Le chiavi dell'indice sono ordinate:

- prima in base al primo attributo in ElencoAttributi
- a pari valore del primo attributo sui valori del secondo attributo
- e così via, in ordine, fino all'ultimo attributo.

Cancellazione: DROP INDEX NomeIndice

Rilasciato sotto licenza: **CC BY-NC 3.0**

Progettazione fisica

Stima del carico applicativo:

- interrogazioni più importanti e loro frequenza
- operazioni di aggiornamento più importanti e loro frequenza
- requisiti sul tempo di risposta per interrogazioni/aggiornamenti importanti

Progettazione fisica: svolta in modo empirico, con un approccio per tentativi

Non esistono metodologie di riferimento

Caratterizzazione del carico applicativo

- Per ogni interrogazione rilevante è necessario definire:
    - relazioni a cui accede
    - attributi da visualizzare
    - attributi coinvolti in selezioni/join
    - grado di selettività delle condizioni di selezione

- Per ogni aggiornamento rilevante è necessario definire:
    - tipo di aggiornamento (insert, delete, update)
    - relazione ed eventuali attributi coinvolti
    - grado di selettività delle condizioni di selezione

Procedimento e scelte da operare

- strutturazione fisica dei file che contengono le tabelle
    - ordinati, non ordinati
- scelta degli attributi da indicizzare
    - pilotata dalla stima del carico applicativo e dal volume dei dati
- per ogni indice definizione del tipo
    - per esempio, hash oppure B-tree
- eventuali variazioni dello schema
    - partizionamenti orizzontali in memoria secondaria
    - denormalizzazione di tabelle
        - utilizzata nei data warehouse

Se il risultato non è soddisfacente: TUNING, aggiungendo e togliendo indici

Rilasciato sotto licenza: **CC BY-NC 3.0**